Electronic Mail from the RFCs: SMTP, POP3 and IMAP4 Servers and Clients, MIME, STARTTLS, and Real-Provider Interoperation

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 Protocols and Message Formats **4**

2.1 SMTP: Commands, Envelope and Replies 4

2.2 POP3 and IMAP4 5

2.3 MIME 5

3 System Architecture **5**

4 Implementation **6**

4.1 SMTP Server: Relay Protection, Dot-Stuffing and Bounces 6

4.2 POP3: Locking and the UPDATE State 7

4.3 IMAP4: Server-Side State and IDLE Push 8

4.4 The POP3 vs IMAP Experiment 8

5 Experiments, Verification and Results **10**

5.1 Running the Module 10

5.2 Automated Verification 10

5.3 Interoperation 11

6 Conclusion **12**

AI Acknowledgement **12**

1 Introduction

Electronic mail is the oldest application on the Internet that is still used by almost everyone. Unlike the web, it is *asynchronous*: the sender and the receiver are rarely online at the same time, so mail is stored on servers between the two. This gives the system two halves with different protocols:

- **Push:** the user agent hands a message to its mail server, and servers hand it on to each other, with the **Simple Mail Transfer Protocol** (**SMTP**, RFC 5321) on TCP port 25 (587 for submission, 465 with implicit TLS).
- **Pull:** the recipient fetches mail from the server with a *mail access protocol*: **POP3** (RFC 1939, port 110/995), which downloads and usually deletes, or **IMAP4** (RFC 3501, port 143/993), which keeps mail, folders and flags on the server so that several devices stay in sync.

SMTP carries only 7-bit text lines, so attachments and non-English text need **MIME** (RFC 2045–2047). Because all three protocols were designed as plaintext, each has an extension to upgrade the same connection to TLS (`STARTTLS`, `STLS`).

1.1 What Was Built

This module implements both sides of e-mail from the RFCs in pure Java 11 with no external libraries:

1. **An SMTP server** with ESMTP extensions, STARTTLS, AUTH PLAIN/LOGIN, dot-stuffing, `Received:` headers, relay protection and bounce messages (DSN).
2. **A POP3 server** with the AUTHORIZATION → TRANSACTION → UPDATE state machine, maildrop locking and STLS.
3. **An IMAP4rev1 server** with folders, FETCH, STORE, server-side SEARCH, COPY/-MOVE, APPEND and **IDLE** push notifications.
4. **SMTP, POP3 and IMAP clients** that work with our servers *and* with real providers (Gmail, Outlook) over TLS.
5. **A MIME library**: multipart attachments, base64, quoted-printable and RFC 2047 encoded subjects.
6. **An optional smart host**: outside mail can be relayed through a real provider such as Gmail.
7. **A live dashboard** on `http://localhost:8083`: the full protocol conversation with every reply code explained, a POP3 vs IMAP experiment on two devices, a MIME viewer and a raw command console.

Table 1: Ports used by the Mail module

Protocol	Plain (+STARTTLS)	Implicit TLS	Real-world ports
SMTP	2525	2465	25, 587 (submission), 465
POP3	1110	2995	110, 995
IMAP	1143	2993	143, 993
Dashboard	HTTP 8083		



Figure 1: The architecture section of the dashboard: user agent, SMTP push, mailbox, POP3/IMAP pull, animated by real traffic.

2 Protocols and Message Formats

2.1 SMTP: Commands, Envelope and Replies

An SMTP transaction separates the **envelope** from the **message**. `MAIL FROM` and `RCPT TO` form the envelope: they decide where the message is delivered and where bounces go. The `From:` and `To:` lines are only headers inside the message, which is why BCC works.

```
S: 220 mail.cse.myuniv.edu ESMTP JU-Mail ready
C: EHLO laptop
S: 250-mail.cse.myuniv.edu Hello laptop [127.0.0.1]
S: 250-SIZE 10485760
S: 250-8BITMIME
S: 250-PIPELINING
S: 250-STARTTLS
S: 250-AUTH PLAIN LOGIN
S: 250-ENHANCEDSTATUSCODES
S: 250 HELP
C: STARTTLS
S: 220 2.0.0 Ready to start TLS          ... TLS handshake on the same TCP connection ...
C: EHLO laptop                           (start over: everything before was plaintext)
C: AUTH PLAIN AGFsaWNlAGFsaWNlMTIz
S: 235 2.7.0 Authentication successful
C: MAIL FROM:<alice@cse.myuniv.edu>
S: 250 2.1.0 Sender <alice@cse.myuniv.edu> OK
C: RCPT TO:<bob@cse.myuniv.edu>
S: 250 2.1.5 Recipient <bob@cse.myuniv.edu> OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Hello ... (headers, blank line, body) ...
C: .
S: 250 2.0.0 OK: queued as 28DF34898CF8
C: QUIT
```

```
S: 221 2.0.0 mail.cse.myuniv.edu closing connection
```

Listing 1: A complete SMTP session with STARTTLS and AUTH (S = server, C = client).

Table 2: SMTP reply-code classes

Code	Meaning
2xx	Success: 220 ready, 221 closing, 235 authenticated, 250 OK.
3xx	Intermediate: 334 AUTH challenge, 354 send the message data.
4xx	Transient failure: try again later.
5xx	Permanent failure: 535 authentication failed, 550 no such user, 552 too large, 554 relay denied.

2.2 POP3 and IMAP4

POP3 has three states. In **AUTHORIZATION** the client logs in (`USER`, `PASS`); in **TRANSACTION** it lists and retrieves messages (`STAT`, `LIST`, `UIDL`, `RETR`, `TOP`) and marks some for deletion (`DELE`, undone by `RSET`); only after `QUIT`, in **UPDATE**, are the marked messages really removed. POP3 knows nothing about folders or read flags.

IMAP keeps everything on the server. Every command carries a **tag** so that replies can be matched and several commands can be in flight; untagged responses (`* 3 EXISTS`) report changes. Flags such as `\Seen` and `\Deleted`, folders, and searches all live on the server, so every device sees the same state.

2.3 MIME

A message with an attachment is a `multipart/mixed` body whose parts are separated by a boundary string. Each part has its own `Content-Type` and `Content-Transfer-Encoding`: text is usually **quoted-printable**, binary data **base64** (about 33% larger, because 3 bytes become 4 characters). Non-ASCII header text uses RFC 2047 encoded words such as `=?UTF-8?B?...?=`.

3 System Architecture

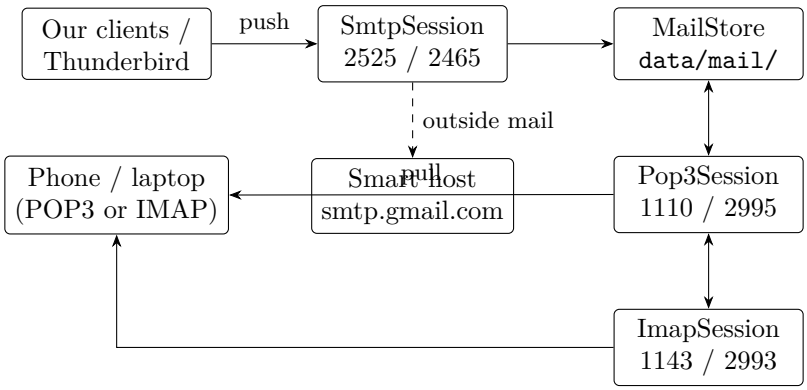


Figure 2: Components of the Mail module.

Table 3: Source packages

Package	Responsibility
`server/`	`Session` (base: listener, one thread per connection, STARTTLS upgrade), `SmtpSession`, `Pop3Session`, `ImapSession`.
`client/`	`LineClient` (base: plain, STARTTLS, implicit TLS), `SmtpClient`, `Pop3Client`, `ImapClient`.
`store/`	`MailStore`: one folder per mailbox, one `.eml` file per message, UIDs and flags.
`mime/`	`Mime`: build and parse multipart messages, base64, quoted-printable, RFC 2047.
`dashboard/`, `core/`	HTTP API and the event hub that records every protocol line.

TLS. On first start `keytool` creates a self-signed certificate (`data/mail-tls.p12`). For STARTTLS the server answers `220 Ready`, then wraps the *existing* socket with `SSLSocketFactory.createSocket(sock, host, port, true)` and performs the handshake. The implicit-TLS ports use an SSL server socket from the start. The clients use the same call in client mode and verify real providers' certificates with Java's default trust store.

4 Implementation

4.1 SMTP Server: Relay Protection, Dot-Stuffing and Bounces

Recipients in our own domain are accepted only if the user exists (`550` otherwise). Recipients in other domains are accepted only from an authenticated user; anyone else gets `554 Relay access denied`, so the server is not an *open relay* that spammers could abuse.

```
private void rcpt(String arg) throws IOException {
    if (from == null) { reply("503 5.5.1 Need MAIL before RCPT"); return; }
    String addr = angle(arg.substring(3).trim());
    int at = addr.indexOf('@');
    String user = at > 0 ? addr.substring(0, at) : addr;
    String dom  = at > 0 ? addr.substring(at + 1) : ctx.store.domain;
    if (dom.equalsIgnoreCase(ctx.store.domain)) {
        if (!ctx.store.userExists(user)) { reply("550 5.1.1 <" + addr + ">: Recipient
    address rejected: User unknown"); return; }
        local.add(user.toLowerCase(Locale.ROOT));
        reply("250 2.1.5 Recipient <" + addr + "> OK");
    } else if (authUser != null) {
        remote.add(addr);
        reply("250 2.1.5 Recipient <" + addr + "> OK (will relay)");
    } else {
        reply("554 5.7.1 <" + addr + ">: Relay access denied");   // no open relay
    }
}
```

Listing 2: RCPT handling in `SmtpSession.java`.

The message ends with a line holding only a dot. A body line that itself starts with a dot is sent with an extra dot by the client (*dot-stuffing*) and the server removes it. The server then prepends `Return-Path:` and `Received:` headers and files the message in each local INBOX.

```
reply("354 End data with <CR><LF>.<CR><LF>");
while ((l = readLine()) != null) {
    if (l.equals(".")) break;                        // end of message
    if (l.startsWith("..")) l = l.substring(1);      // undo dot-stuffing
    if (body.size() < MAX_SIZE) body.write((l + "\r\n").getBytes(StandardCharsets.UTF_8));
}
String received = "Return-Path: <" + from + ">\r\n"
```

```
        + "Received: from " + helo + " (" + peer + ") by mail." + ctx.store.domain + " (JU-
    Mail) with "
        + (authUser != null ? "E" : "") + "SMTP" + (secure ? "S" : "") + (authUser != null ?
     "A" : "")
        + " id " + qid + "; " + Mime.rfc822Date(new Date()) + "\r\n";
for (String u : local) ctx.store.append(u, "INBOX", msg, null);
for (String r : remote) relayOrBounce(r, msg, subject);
reply("250 2.0.0 OK: queued as " + qid);
```

Listing 3: Reading DATA, undoing dot-stuffing and adding trace headers.

Outside mail. A real mail server would look up the recipient domain's MX record in DNS and connect to it on port 25. A lab PC cannot do that (ISPs block port 25 and providers reject mail from unknown home addresses), so without further configuration the server writes a **delivery status notification** back to the sender's INBOX. If `data/smarthost.properties` names a provider, the server instead logs in to it as a client over STARTTLS with AUTH and hands the message on, exactly as Outlook or Thunderbird sends through Gmail. With a Gmail app password configured, a letter posted from the dashboard was delivered to a real Gmail inbox.



Figure 3: Sending a letter: the colour-coded transcript with every reply code explained.

4.2 POP3: Locking and the UPDATE State

The maildrop is locked for the whole session, so a second POP3 session for the same user is refused. `DELE` only records the message number; the deletion happens in the UPDATE state after `QUIT`.

```
private void quit() throws IOException {
    if (state == State.TRANSACTION) {
        MailStore.Folder inbox = ctx.store.folder(user, "INBOX");
        for (int n : deleted) {                          // only now are DELE marks applied
            MailStore.Message m = drop.get(n - 1);
            Set<String> f = new TreeSet<>(m.flags);
            f.add("\\Deleted");
```

```
            ctx.store.setFlags(user, inbox, m, f);
        }
        int removed = ctx.store.expunge(user, inbox).size();
        ok("Bye: " + removed + " message(s) deleted", "UPDATE state: messages removed, lock
    released");
    } else ok("Bye", "");
}
```

Listing 4: The UPDATE state in `Pop3Session.java`.

4.3 IMAP4: Server-Side State and IDLE Push

The IMAP server supports tags, literals and sequence/UID sets; `LIST`, `SELECT`/`EXAMINE` and `STATUS`; `FETCH` of flags, envelope, body structure and partial bodies (with `BODY.PEEK` that does not set `\Seen`); `STORE`, `SEARCH`, `COPY`/`MOVE` (UIDPLUS), `APPEND`, `EXPUNGE` and folder management. Changes made by one session are reported to the others as untagged `EXISTS`, `EXPUNGE` and `FETCH FLAGS` responses.

With **IDLE** (RFC 2177) the client stops polling: the server keeps the connection open and pushes changes until the client sends `DONE`.

```
private void idle(String tag) throws IOException {
    send("+ idling", "server will now push changes in real time; client ends with DONE");
    idling = true;
    Thread w = new Thread(() -> {
        while (idling) {
            try { Thread.sleep(250); sync(true); } catch (Exception e) { return; }   // push
     * n EXISTS etc.
        }
    }, id + "-idle");
    w.setDaemon(true);
    w.start();
    String l;
    while ((l = readLine()) != null) if (l.equalsIgnoreCase("DONE")) break;
    idling = false;
    tagged(tag, "OK IDLE terminated");
}
```

Listing 5: IDLE in `ImapSession.java`.

4.4 The POP3 vs IMAP Experiment

The dashboard gives Bob two devices, a phone and a laptop, that share one mailbox. Each can use IMAP, POP3 in keep mode, or POP3 in delete mode.

Table 4: Observed behaviour of the two-device experiment

Setting	Result
Phone on POP3 (delete), laptop on IMAP	The phone downloads everything and the server column becomes empty; the laptop then sees no mail and the table reports that the devices disagree.
Phone on POP3 (keep)	Letters stay on the server; UIDL tells the phone which ones it already has. Read state is not shared.
Both on IMAP	Opening a letter on the laptop sets `\Seen` on the server, and the phone shows it as read. Moving to a folder works only with IMAP.
IMAP with IDLE	A new letter appears immediately as a pushed `* n EXISTS`.

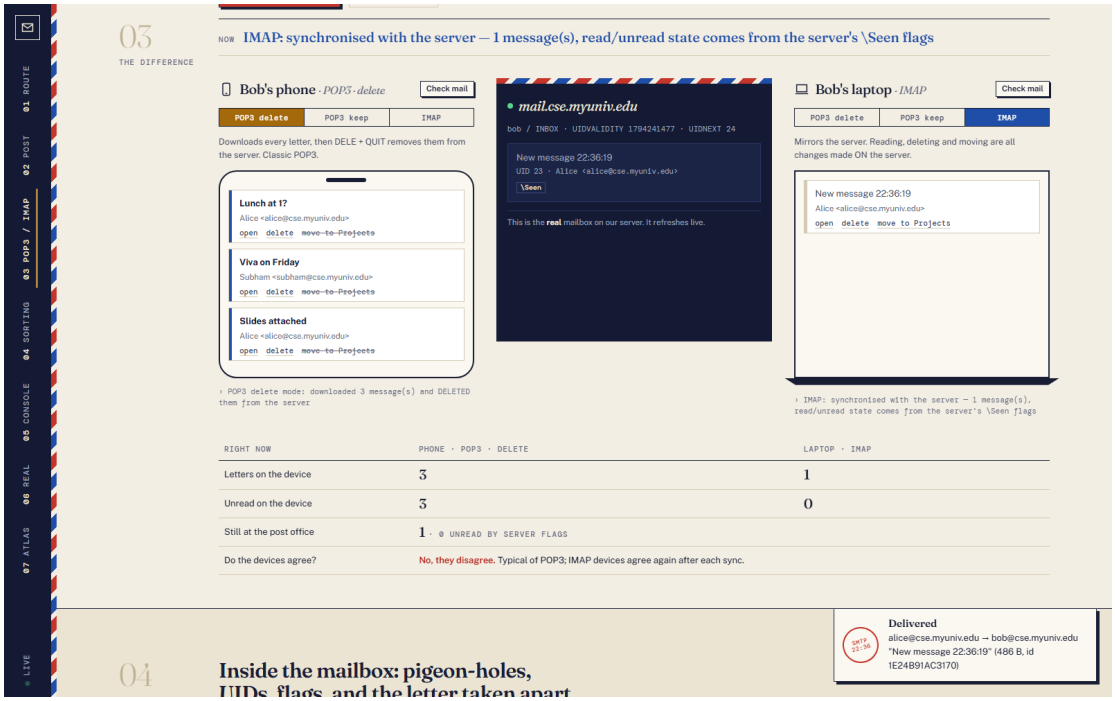


Figure 4: The POP3 vs IMAP experiment: what each protocol does to the mailbox on the server.

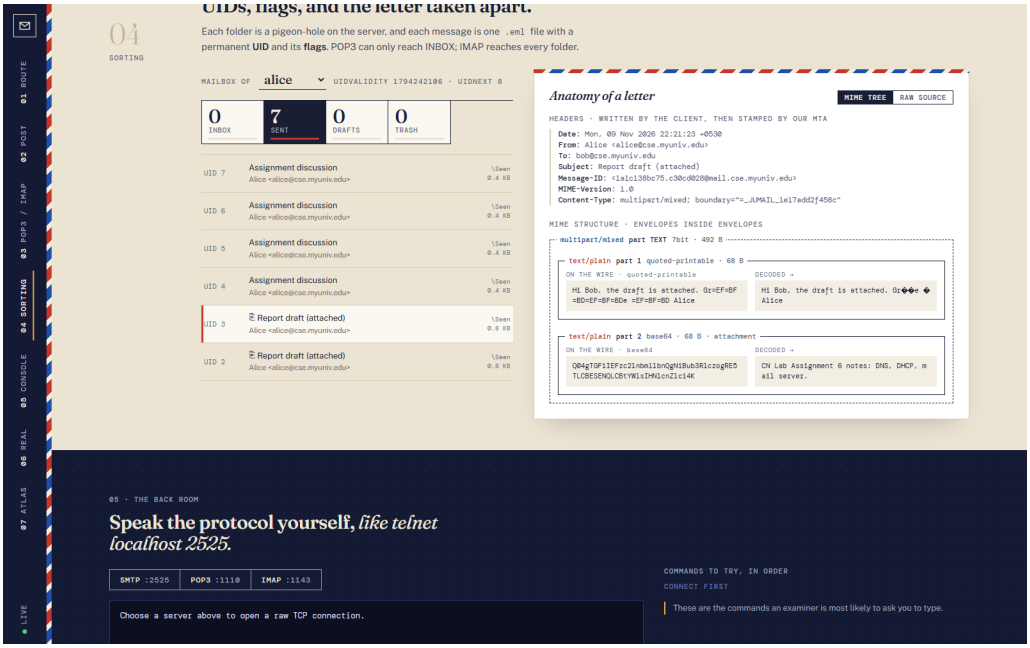


Figure 5: Inside the mailbox: folders, UIDs, flags, and the MIME tree of a message with an attachment.

5 Experiments, Verification and Results

5.1 Running the Module

```
cd Mail
run.bat                  # SMTP, POP3, IMAP servers and dashboard (http://localhost:8083)
run.bat --verify         # 39 automated end-to-end checks
python interop_test.py   # Python's smtplib, poplib and imaplib against our servers (20
    checks)

telnet localhost 2525    # then: EHLO, MAIL FROM, RCPT TO, DATA, ".", QUIT
```

Listing 6: Starting the servers, running the checks, and talking SMTP by hand.

5.2 Automated Verification

All **39 checks** of `Verify.java` pass. They drive the real servers with the module's own clients:

Table 5: Selected verification checks (all passed)

`multipart/mixed` with two parts; the base64 attachment decodes to the original bytes; RFC 2047 subject round-trips
EHLO advertises STARTTLS, AUTH, SIZE, 8BITMIME; STARTTLS upgrade, then a new EHLO no longer offers STARTTLS
Unauthenticated relay refused (554); unknown local user rejected (550); wrong password gives 535
Dot-stuffing undone; server added `Return-Path` and `Received` headers
Authenticated user may relay; with no smart host a bounce (DSN) is delivered to the sender
POP3: UIDL gives permanent ids; a second session is refused (maildrop locked); DELE only marks, QUIT removes
IMAP: `BODY.PEEK` does not set `\Seen`, `BODY[]` does; device B sees device A's flag change
IMAP IDLE: new mail is pushed to device B; UID MOVE, APPEND to Sent, CREATE folder
A POP3 delete on one device shows up as EXPUNGE on the IMAP device
SMTPS, POP3S and IMAPS (implicit TLS) and STARTTLS/STLS all work

5.3 Interoperation

1. **Standard libraries against our servers.** `interop_test.py` uses Python's `smtplib`, `poplib` and `imaplib`, which we did not write, to send, list, fetch, search and append; all 20 checks pass. This shows the servers follow the RFCs rather than only our own client.

2. **Real mail applications.** Thunderbird configured with IMAP `localhost:1143` and SMTP `localhost:2525` sends and reads mail; read flags stay in sync with the dashboard.

3. **Our clients against real providers.** The dashboard's clients connect to Gmail and Outlook over TLS (IMAP 993, POP3 995, SMTP 587 with STARTTLS) using an app password.

4. **Real delivery through a smart host.** With Gmail configured as smart host, a letter posted to an external Gmail address from the dashboard arrived in that Gmail inbox.



Figure 6: The same clients connected to a real provider.

6 Conclusion

1. **Push and pull are different problems.** SMTP moves mail between servers and must decide whom it may relay for; POP3 and IMAP give a user access to a stored mailbox. Building all three showed why they are separate protocols.
2. **Where state lives decides the user experience.** POP3 in delete mode left a second device with nothing; IMAP kept folders and read flags on the server and, with IDLE, pushed new mail without polling.
3. **Plaintext protocols can be upgraded in place.** STARTTLS and STLS wrap the existing TCP connection in TLS, after which the session starts over; credentials were never sent before the upgrade.
4. **The servers are real.** Python's standard mail libraries, Thunderbird and Windows `telnet` all work with them, and the same clients work with Gmail, so the implementation follows the RFCs rather than a private convention.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.